

Verifiable Observability: Project Overview

Real-Time Behavioral Verification for Multi-Turn LLM Agents

1. The Problem: The "Black Box" Dilemma

As AI agents become more autonomous, they are being deployed in high-stakes environments like **Finance**, **Healthcare**, and **Code Execution**. However, Large Language Models (LLMs) are probabilistic and prone to hallucination or "behavioral drift."

If an AI agent executes a \$50,000 trade, accesses a patient's protected medical record, or runs a shell command on a host server, organizations cannot simply trust the agent's output. They need a way to **mathematically prove** that the agent followed Standard Operating Procedures (SOPs) and enforce hard safety constraints *before* any action is taken.

2. Our Solution: Verifiable Observability

Verifiable Observability is a runtime monitoring and enforcement framework. Instead of letting the agent act freely, the framework wraps the agent in a strict verification layer. It continuously observes the agent's reasoning, grades its compliance against predefined rules, and uses a deterministic firewall to block unsafe actions instantly.

The 3-Layer Safety Architecture

Every time the agent attempts an action, it must pass through three verification layers:

1. **Strategy Profiler:** When a user submits a task (e.g., *"Transfer \$50,000"*), the Profiler assigns it a Task Type and Risk Tier (LOW, MEDIUM, HIGH) to set a behavioral baseline.
2. **Rule Bank:** This is the database of Standard Operating Procedures (SOPs). It grades the agent's reasoning to ensure it aligns with expected protocols (e.g., *"If transferring funds, the agent MUST check the balance first"*).
3. **Constraint Compliance Monitor (CCM):** The strict firewall. It actively intercepts API calls before they reach the real world and outputs one of three decisions:
 - ✓ **ALLOW:** The action is safe. A cryptographic execution ticket is generated to prevent tampering.
 - ⚠ **FLAG:** The action violates a soft constraint (e.g., unusually large trade). It executes but is escalated for human review.
 - 🛑 **BLOCK:** The action violates a hard constraint (e.g., prompt injection attempt, missing HIPAA logs). The action is stopped immediately.

3. How The Project Works in Practice

Cross-Domain Applications

The framework is highly modular and has been fully implemented across three high-stakes domains:

- **Finance:** Hard-blocks transfers > \$50,000 without prior compliance approval. Flags unusually large share quantities.
- **Healthcare:** Blocks access to Protected Health Information (PHI) unless a strict HIPAA audit log is written first.
- **Code Execution:** Blocks execution of shell commands in a sandbox without prior static analysis passes.

Real-World Walkthrough

1. **User Prompt:** *"Execute a massive transfer of \$100,000 from ACC-001."*
2. **LLM Reasoning:** The agent decides to skip the compliance review and attempts to call `execute_transfer` for \$100,000.
3. **CCM Interception:** The `FinanceCCM` catches the API call before it executes.
4. **Outcome:** The CCM checks its rules, sees the hard limit of \$50,000 without approval, and returns **BLOCK**. The agent is stopped, and the dashboard turns red.

Measuring Success: The Dual-Metric System

The system tracks the agent's health over time using two mathematically bounded metrics (0.0 to 1.0):

- **RCR (Reasoning Consistency Ratio):** Did the agent's internal thoughts align with the SOPs? (Detects degraded logic).
- **CCR (Constraint Compliance Ratio):** Did the agent's physical actions violate any safety limits? (Detects actual violations).

Using advanced statistical models (CUSUM Changepoint Detection and Beta-Binomial models), the framework tracks these metrics over time. If an agent begins to "drift" and forget its safety rules, the system detects the anomaly and flags the agent for retraining *before* a catastrophic failure occurs.

The Verifier Tax (Safety vs. Utility)

A major research contribution of this project is quantifying the "verifier tax." By running thousands of simulated trajectories, we generated a Pareto curve that proves our default system settings block 100% of adversarial attacks while maintaining an 80% task completion rate for benign operations.